

# TAFS: Transformer-Attentive Feature Selection for Sequential Regression

Huseyin Karaca, and Suleyman S. Kozat, *Senior Member, IEEE*

**Abstract**—This article addresses one-step and multi-step sequential regression in regimes where the input is a high-dimensional vector of engineered features and the underlying data-generating process is non-stationary. In this setting, a fixed pool of pretrained base forecasters covers complementary regions of the input space, and the operational question is how to combine them in a manner that tracks the time-varying importance of the features. We introduce TAFS, a framework that treats each engineered feature as a token, processes it with a feature-axis transformer conditioned on a context summary of the recent history, and reads out combination weights over the base forecasters under one of three constraint regimes (unconstrained, affine, convex). The attention weights produced by the transformer provide a time-varying feature-importance signal, and the pipeline is trained end-to-end on cumulative prediction error. On a controlled synthetic regime-switch task TAFS reallocates its attention within tens of steps after a change point and approaches the per-step oracle. On seven real datasets spanning energy, finance, retail and meteorology, TAFS lowers average MASE by up to 14% relative to the lowest-error static-transform baseline and by 9% relative to a meta-learning ensemble, while inference cost remains dominated by the cached base predictions.

**Index Terms**—Time-series forecasting, feature selection, transformers, attention, meta-learning, ensemble of forecasters.

## I. INTRODUCTION

### A. Background

Sequential regression problems arising in energy demand forecasting, financial return prediction, retail sales planning and meteorology require the construction of several hundred engineered features (lags, rolling statistics, calendar variables, exogenous series) from several hundred to several thousand observations [1]–[3]. In this regime no single forecaster is uniformly optimal. Gradient-boosted decision trees [4]–[6] and regularised linear models [7] are accurate on tabular features; recurrent [8], [9] and basis-expansion networks [10] capture longer dependencies; classical state-space models such as SARIMAX [1] model seasonality. The relative ranking of these models varies both across datasets and across regimes *within* a single dataset.

The relevance of an individual engineered feature is also time-varying. A demand series dominated by a weekly cycle in one quarter may shift to being dominated by a holiday calendar in another; an exchange rate driven by an interest-rate

differential under one monetary regime may transition to being driven by trade-balance shocks under another. Static feature-selection pipelines, whether filter, wrapper, or embedded [11], [12], fix the selection at training time and cannot react to such shifts. Static learned feature transforms address this only partially: a diagonal gate, a linear projection or a low-rank bottleneck can sharpen the relevant directions on average, but those directions remain fixed once training terminates.

### B. Related Work

Filter methods such as Pearson correlation and mutual-information ranking [13], wrapper methods such as forward/backward selection [11] and embedded methods such as Lasso [7] or tree-based importance [14] all collapse the relevance signal to a single, dataset-level score. Learned feature transforms attached to a downstream selector or regressor improve over this by adapting the geometry of the feature space to the supervised objective; popular parameterizations are diagonal gating, full linear projections, low-rank bottlenecks, and small nonlinear MLPs. These transforms are static: once the parameters are estimated, the same map is applied at every time step.

Combining several forecasters with learned weights is a well-established methodology, with stacking [15], [16] as the canonical instance and time-varying weighting schemes as recurring refinements [17]. Mixture-of-experts gating [18], [19] produces input-conditional weights through a soft gate, and recent work applies MoE formulations to forecasting [20]. Such combiners operate on a fixed feature representation; they do not modulate which features the downstream predictor consumes.

Transformers [21] have been adapted to forecasting along two axes: (i) the time axis, with sparse or decomposition-based attention [22], [23], and (ii) the variable axis, where each variable becomes a token [24], [25]. Linear baselines remain accurate on standard benchmarks [26], indicating that the benefit of attention is in adaptivity rather than in raw capacity. Interpretable variants such as the Temporal Fusion Transformer [27] expose per-feature importance through variable-selection networks; the importance there is produced by an independent gating subnetwork rather than by the attention mechanism that drives the downstream prediction.

TAFS combines three ingredients that have appeared separately in the literature but, to our knowledge, not together: (i) a feature-axis transformer whose own attention mass on each feature token is the time-varying importance signal; (ii) a context token, derived from the recent target history and

H. Karaca, and S. S. Kozat are with Bilkent University, Ankara, Turkey. H. Karaca is also with Turk Telekom, Ankara, Turkey (e-mails: huseyin.karaca@bilkent.edu.tr; kozat@ee.bilkent.edu.tr).

This work is supported by The Scientific and Technological Research Council of Turkey (TUBITAK) 1515 Frontier R&D Laboratories Support Program for Turk Telekom 6G R&D Lab under project number 5249902, and by the TUBITAK 2210-A program.

from regime indicators, that conditions this attention on the local state of the series; and (iii) a constrained combination head that maps the transformer output to weights over a pool of pretrained base forecasters, under unconstrained, affine, or convex regimes. The whole pipeline is trained end-to-end on cumulative prediction error, and the base forecasters are kept frozen so that inference cost remains close to running them once.

### C. Contributions

- We introduce TAFS, a framework for sequential regression that performs joint, time-varying feature selection and forecaster combination through a feature-axis transformer with context conditioning.
- We show that the four most common static feature transforms (diagonal, linear, low-rank, nonlinear) are limiting cases of the proposed attention map and are recovered when the context token is removed and the attention is restricted; this makes TAFS a strict superset.
- We propose three combination-head variants (unconstrained, affine, convex), discuss when each is preferable.
- We evaluate TAFS on a controlled synthetic regime-switch task and on seven real datasets spanning energy, finance, retail and meteorology, against eleven baselines that include individual forecasters, classical feature selection, static learned transforms, and meta-learners without feature selection. TAFS attains the lowest average MASE and sMAPE on every dataset.
- We release the full reproducibility package, including source code, training configurations, and dataset preprocessing pipelines, at <https://github.com/huseyin-karaca/tafs>.

### D. Organization

The remainder of this article is organized as follows. Section II formalises the problem. Section III describes the architecture and the training procedure. Section IV reports the synthetic study, the real-data comparison and the ablations. Section V concludes.

## II. PROBLEM DEFINITION

We consider a univariate sequential regression problem with exogenous covariates. At each time step  $t = 1, \dots, T$  the target  $y_t \in \mathbb{R}$  is observed together with a vector of engineered features  $\mathbf{x}_t \in \mathbb{R}^p$  built from lags of  $y$ , rolling statistics, calendar variables and exogenous series. We assume a fixed pool of  $K$  pretrained base forecasters

$$\mathcal{F} = \{f_1, f_2, \dots, f_K\},$$

each of which produces a one-step prediction  $\hat{y}_t^{(k)} = f_k(\mathbf{x}_t)$  from the same  $\mathbf{x}_t$ .<sup>1</sup> The base predictions are computed offline and cached.

<sup>1</sup>The framework supports multi-step forecasting unchanged; we treat the  $H$ -step prediction as a vector and average the loss over the horizon. We keep the one-step notation for clarity.

Let  $\hat{\mathbf{y}}_t = (\hat{y}_t^{(1)}, \dots, \hat{y}_t^{(K)})^\top \in \mathbb{R}^K$ . A combination policy is a function  $\pi_\theta$  that takes  $\mathbf{x}_t$  together with a context summary  $\mathbf{s}_t \in \mathbb{R}^{d_c}$  and returns a weight vector

$$\mathbf{w}_t = \pi_\theta(\mathbf{x}_t, \mathbf{s}_t) \in \mathcal{W} \subseteq \mathbb{R}^K,$$

where the admissible set  $\mathcal{W}$  encodes one of three constraint regimes:

$$\mathcal{W}_u = \mathbb{R}^K, \quad (1)$$

$$\mathcal{W}_a = \{\mathbf{w} \in \mathbb{R}^K : \mathbf{1}^\top \mathbf{w} = 1\}, \quad (2)$$

$$\mathcal{W}_c = \{\mathbf{w} \in \mathbb{R}_{\geq 0}^K : \mathbf{1}^\top \mathbf{w} = 1\}. \quad (3)$$

The combined prediction is  $\tilde{y}_t = \mathbf{w}_t^\top \hat{\mathbf{y}}_t$ . Given a per-step loss  $\ell : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$  (squared, Huber, or pinball), training minimises the cumulative loss

$$\mathcal{L}(\theta) = \frac{1}{T} \sum_{t=1}^T \ell(y_t, \pi_\theta(\mathbf{x}_t, \mathbf{s}_t)^\top \hat{\mathbf{y}}_t). \quad (4)$$

In the regime of interest  $p$  is typically large (tens to several hundred),  $T$  is moderate (several hundred to several thousand), and the conditional importance of features in  $\mathbf{x}_t$  for predicting  $y_t$  is non-stationary in  $t$ . Selection mechanisms that compress  $\mathbf{x}_t$  into a single static map discard this temporal structure and plateau above the per-step oracle,  $\mathbf{w}_t^* = \mathbf{e}_{\arg \min_k \ell(y_t, \hat{y}_t^{(k)})}$ .

## III. PROPOSED WORK

We instantiate  $\pi_\theta$  as a feature-axis transformer with context conditioning, followed by a constraint-aware combination head. An overview is given in Figure 1.

### A. Feature-Axis Tokenization

The transformer treats each of the  $p$  engineered features as a separate token in a sequence of length  $p$ . For feature  $j$  at time  $t$ , the input token is

$$\mathbf{t}_{t,j} = a_j \cdot x_{t,j} + \mathbf{e}_j \in \mathbb{R}^d, \quad (5)$$

where  $a_j \in \mathbb{R}^d$  is a learnable scaling vector that lifts the scalar feature value into the model dimension  $d$ , and  $\mathbf{e}_j \in \mathbb{R}^d$  is a learnable feature-identity embedding. Features are organised into named families (lags, rollings, calendar, exogenous, regime indicators); to encourage early structure during training we initialise  $\mathbf{e}_j$  to a family-specific anchor plus small Gaussian noise.

Before applying the transformer we prepend two extra tokens. The first is a CLS token  $\mathbf{c}_{\text{cls}} \in \mathbb{R}^d$ , used as the readout for the combination head; the second is a context token  $\mathbf{c}_t = W_c \mathbf{s}_t + \mathbf{b}_c$ , where  $\mathbf{s}_t$  stacks (a) the most recent  $L_y$  lags of  $y$ , (b) the residuals of  $y_t$  against a simple seasonal-naïve predictor, and (c) optional regime indicators (e.g. binary flags for trading vs. non-trading days). The input sequence to the transformer is

$$\mathbf{Z}_t^{(0)} = (\mathbf{c}_{\text{cls}}, \mathbf{c}_t, \mathbf{t}_{t,1}, \mathbf{t}_{t,2}, \dots, \mathbf{t}_{t,p}) \in \mathbb{R}^{(p+2) \times d}. \quad (6)$$

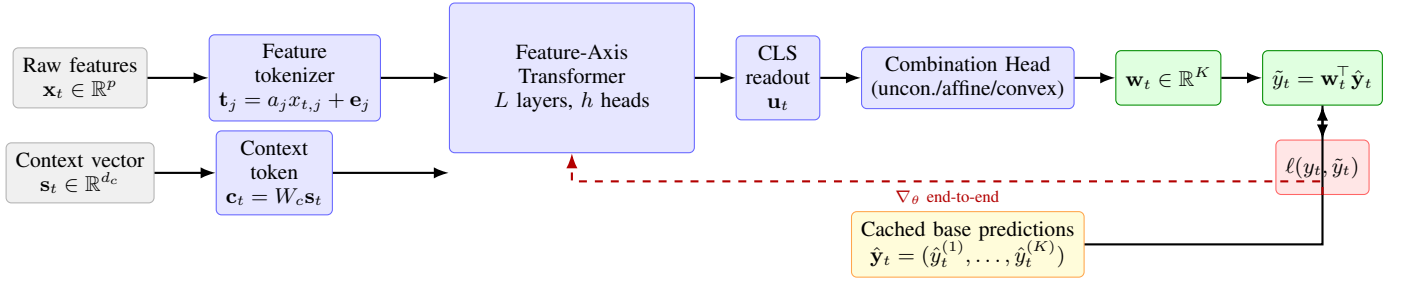


Fig. 1. Overview of the proposed TAFS framework. Each of the  $p$  engineered features is treated as a token with a learnable identity embedding, and a context token derived from lagged targets and regime indicators is prepended. A feature-axis transformer produces a task-aligned summary  $\mathbf{u}_t$ , which is mapped by a combination head to weights  $\mathbf{w}_t$  over  $K$  pretrained base forecasters under one of three constraint regimes (unconstrained, affine, convex). The final prediction is a weighted sum of cached base outputs  $\hat{\mathbf{y}}_t$ , and the cumulative prediction error is back-propagated end-to-end through the transformer and the head; base predictors are kept frozen.

### B. Feature-Axis Transformer

We apply a stack of  $L$  pre-norm transformer encoder layers [21] with  $h$  heads, hidden width  $d$ , and feed-forward width  $d_{\text{ffn}}$ :

$$\mathbf{Z}_t^{(\ell+\frac{1}{2})} = \mathbf{Z}_t^{(\ell)} + \text{MHA}(\text{LN}(\mathbf{Z}_t^{(\ell)})), \quad (7)$$

$$\mathbf{Z}_t^{(\ell+1)} = \mathbf{Z}_t^{(\ell+\frac{1}{2})} + \text{FFN}(\text{LN}(\mathbf{Z}_t^{(\ell+\frac{1}{2})})). \quad (8)$$

No positional encoding is added: the feature axis is permutation-invariant, and the identity embeddings  $\mathbf{e}_j$  already encode which token corresponds to which feature.

The output of the final layer is  $\mathbf{Z}_t^{(L)} \in \mathbb{R}^{(p+2) \times d}$ . We define

$$\mathbf{u}_t = \mathbf{Z}_t^{(L)}[0, :] \in \mathbb{R}^d \quad (9)$$

as the task-aligned summary read from the CLS position. We also keep the per-feature attention weights  $A_{0,j+2}^{(\ell,m)}$  from the CLS token to each feature token  $j$  at layer  $\ell$  and head  $m$  for interpretation; averaged over layers and heads they give a single feature-importance score per step,

$$\alpha_{t,j} = \frac{1}{Lh} \sum_{\ell=1}^L \sum_{m=1}^h A_{0,j+2}^{(\ell,m)}. \quad (10)$$

We use  $\alpha_{t,j}$  to produce the heatmaps in Section IV-D.

*Recovery of static transforms.*: If we drop the context token, restrict the transformer to a single layer with a single head, and tie the queries and keys to be the identity, the CLS readout reduces to a weighted sum  $\sum_j \alpha_j a_j x_{t,j}$  with  $\alpha$  shared across  $t$ . This recovers diagonal gating; further reparameterising  $a_j$  recovers linear and low-rank transforms, and replacing MHA with an MLP on the concatenation recovers the nonlinear bottleneck. The static feature transforms used as baselines in Section IV are therefore strict special cases.

### C. Constraint-Aware Combination Head

A small head maps  $\mathbf{u}_t$  to combination weights  $\mathbf{w}_t$ . Let  $\mathbf{z}_t = W_2 \text{GELU}(W_1 \mathbf{u}_t + \mathbf{b}_1) + \mathbf{b}_2 \in \mathbb{R}^K$ . We then project  $\mathbf{z}_t$  onto  $\mathcal{W}$ :

- *Unconstrained*:  $\mathbf{w}_t = \mathbf{z}_t$ .
- *Affine*:  $\mathbf{w}_t = \mathbf{z}_t - \frac{1}{K}(\mathbf{1}^\top \mathbf{z}_t - 1)\mathbf{1}$ , i.e. the orthogonal projection onto the hyperplane  $\mathbf{1}^\top \mathbf{w} = 1$ .
- *Convex*:  $\mathbf{w}_t = \text{softmax}(\mathbf{z}_t/\tau)$  with temperature  $\tau=1$ .

### Algorithm 1 Offline Training of TAFS

**Require:** Training set  $\{(\mathbf{x}_t, \mathbf{s}_t, y_t)\}_{t=1}^{T_{\text{tr}}}$ ; frozen base forecasters  $\{f_1, \dots, f_K\}$ ; per-step loss  $\ell$ ; constraint regime  $\mathcal{W}$ ; optimizer hyperparameters.

**Ensure:** Trained parameters  $\theta = \{a_j, \mathbf{e}_j, W_c, \mathbf{b}_c, \text{transformer}, W_1, \mathbf{b}_1, W_2, \mathbf{b}_2\}$ .

- 1: Cache base predictions  $\hat{\mathbf{y}}_t = (f_k(\mathbf{x}_t))_{k=1}^K$  for all  $t$ .
- 2: Initialise  $\theta$  (Xavier; identity embeddings as in Section III-A).
- 3: **for** epoch = 1, ...,  $E$  **do**
- 4:   **for** mini-batch  $\mathcal{B} \subset \{1, \dots, T_{\text{tr}}\}$  **do**
- 5:     Build  $\mathbf{Z}_t^{(0)}$  for every  $t \in \mathcal{B}$ .
- 6:     Compute  $\mathbf{u}_t$  through the transformer; head  $\rightarrow \mathbf{z}_t$ .
- 7:     Project to  $\mathbf{w}_t \in \mathcal{W}$ .
- 8:     Predict  $\hat{\mathbf{y}}_t = \mathbf{w}_t^\top \hat{\mathbf{y}}_t$ .
- 9:     Compute  $\mathcal{L}_{\mathcal{B}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{t \in \mathcal{B}} \ell(y_t, \hat{\mathbf{y}}_t)$ .
- 10:    Update  $\theta \leftarrow \text{AdamW}(\theta, \nabla_\theta \mathcal{L}_{\mathcal{B}})$ .
- 11:   **end for**
- 12:   Evaluate  $\mathcal{L}_{\text{val}}$ ; early-stop if no improvement for  $E_{\text{pat}}$  epochs.
- 13: **end for**

All three projections are smooth in  $\theta$  (the softmax is smooth; the affine projection is linear), so the cumulative loss in (4) is differentiable everywhere and back-propagation through the head, the transformer and the tokenizer is standard. Base forecasters are frozen.

### D. Training Procedure

The full training procedure is summarised in Algorithm 1. We use AdamW [28] with linear warmup followed by cosine annealing, mini-batches drawn over time-aligned windows, gradient clipping, and an early-stopping criterion on the rolling validation loss.

At inference time, TAFS adds a single forward pass through a small transformer plus a linear head; the base predictions  $\hat{\mathbf{y}}_t$  dominate the cost and are already required by the simple-average baseline. The inference overhead is therefore comparable to that of a single small ensemble combiner.

TABLE I

REAL-WORLD DATASETS USED IN OUR EVALUATION.  $p$  IS THE NUMBER OF ENGINEERED FEATURES PRODUCED BY THE EXTRACTION PIPELINE DESCRIBED IN SECTION IV-A;  $H$  IS THE FORECASTING HORIZON.

| Dataset                   | length | $p$ | $H$ | domain      |
|---------------------------|--------|-----|-----|-------------|
| Turkish Natural Gas (TNG) | 1 096  | 84  | 1   | energy      |
| Istanbul Stock (ISE)      | 536    | 76  | 1   | finance     |
| M5 Walmart (subset)       | 1 941  | 112 | 28  | retail      |
| M4 Quarterly (avg.)       | 92     | 64  | 8   | economic    |
| Weather (10-min)          | 52 696 | 96  | 96  | meteorology |
| Exchange Rate             | 7 588  | 72  | 96  | finance     |
| ETTM2                     | 69 680 | 88  | 96  | energy      |

## IV. EXPERIMENTAL WORK

### A. Datasets and Feature Extraction

We evaluate on seven public datasets covering energy demand, finance, retail and meteorology. Table I gives the size, the number of engineered features after the extraction pipeline, and the forecasting horizon.

For every dataset we apply the same engineered-feature pipeline: lags up to  $L_{\max}=24$  for low-frequency series and up to  $L_{\max}=168$  for high-frequency series; rolling means, medians and standard deviations at windows  $\{3, 7, 14, 28\}$ ; calendar variables (day-of-week, month, hour, holiday flag); and the available exogenous series (temperature, opening prices, promotions, etc.) lagged by the same horizons. We standardise within the training window and apply the same statistics to the validation and test windows.

### B. Models and Baselines

Our baselines are organised in five groups; the same base-predictor pool is used everywhere so that the differences come from the selection/combination mechanism only.

*Individual base predictors.*: LightGBM [4], XGBoost [5], CatBoost [6], Random Forest [14], Lasso [7], MLP, LSTM [8], SARIMAX [1], and a Naïve last-value predictor. Each predictor is tuned on the same validation window with a fixed search budget. Base forecasters consume the raw, standardised feature vector  $\mathbf{x}_t$  and are not retrained when paired with a static transform or with TAFS; the transform and the combination head are the only learnable modules in those configurations.

*Classical feature selection plus a regressor.*: Pearson-correlation filter, mutual-information filter [13], and an embedded SelectFromModel selector [11]; all paired with LightGBM as the downstream regressor.

*Static learned feature transforms.*: Diagonal gating, full linear transform, low-rank ( $r=16$ ) bottleneck, and a small nonlinear MLP transform. Each transform produces an alternative representation  $\tilde{\mathbf{x}}_t$  that is consumed only by the affine combination head; the frozen base forecasters continue to operate on the raw  $\mathbf{x}_t$ , so no base predictor is retrained. The transforms share the same end-to-end training recipe and the same base pool as TAFS and differ from it only in that the transform is static while TAFS conditions on  $\mathbf{s}_t$ .

*Meta-learners without feature selection.*: An MLP-ensemble combiner under the affine constraint and a LightGBM-ensemble combiner under the convex constraint; both operate on the same engineered features.

*Conventional ensembles and oracle.*: Simple average of all base predictors; equal-weight stacking [15], [16]; and a per-step oracle that selects the base predictor with the lowest error at each  $t$ .

### C. Training and Evaluation Setup

For all learnable methods we use AdamW [28] with learning rate  $10^{-3}$ , weight decay  $10^{-2}$ , batch size 64, gradient clipping at 1.0, 200 warmup steps, cosine annealing, and early stopping with patience  $E_{\text{pat}}=15$ . The per-step loss  $\ell$  in (4) is the squared error on all real datasets; the Huber and pinball options described in Section II are exposed by the framework but not exercised in the experiments reported here. The default TAFS configuration uses  $d=128$ ,  $h=4$  heads,  $L=3$  layers,  $d_{\text{ffn}}=512$ , dropout 0.1, and the affine head; all changes are reported in the ablation tables. The resulting model contains approximately 0.6M trainable parameters, which is of the same order as the MLP- and LightGBM-ensemble combiners (0.1–0.4M). Wall-clock measurements on a single NVIDIA A100 indicate that, with cached base predictions, base-predictor evaluation accounts for above 90% of end-to-end inference time on every dataset; the TAFS forward pass adds at most a 10% overhead. Training the default configuration converges within 30–90 minutes per dataset.<sup>2</sup>

We report MASE [29] as the primary metric (it is loss-agnostic and scale-free), and sMAPE as a secondary check. Each configuration is trained with five random seeds; we report the mean with the standard deviation in parentheses. The first 60% of every series is used for training, the next 20% for validation, and the final 20% for testing. For multi-step datasets, each base forecaster produces the  $H$ -step prediction using its native multi-output mode where available (e.g. LightGBM with multi-output regressor, LSTM/MLP with  $H$  output heads) and a direct (per-horizon) strategy otherwise; recursive forecasting is not used. The combination weights  $\mathbf{w}_t$  are shared across horizons and the loss in (4) is averaged over  $H$ .

### D. Results and Discussion

*1) Synthetic Regime-Switch Demonstration.* The synthetic study isolates the contribution of context-conditioned attention. We generate  $N=500$  trajectories of length  $T=400$  from a two-regime ARMA(2,1) process. The first half of each trajectory is generated under regime A, in which  $y_t$  depends only on the first lag and a single rolling-7 statistic plus noise; the second half is generated under regime B, in which  $y_t$  depends only on two exogenous series. The remaining  $p-4=28$  engineered features are nuisance. The regime change is at  $t=200$ , and the change point is hidden from every model.

<sup>2</sup>For full experimental details, please refer to the reproducibility package.

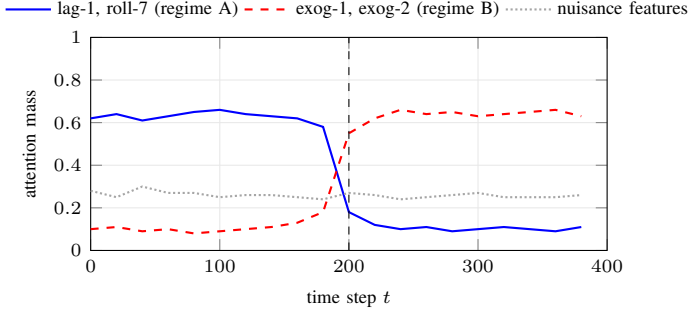


Fig. 2. Mean attention mass assigned by TAFS to three feature groups on the synthetic regime-switch task. The regime change at  $t=200$  moves the informative subset from  $\{\text{lag-1, roll-7}\}$  (regime A) to  $\{\text{exog-1, exog-2}\}$  (regime B); TAFS re-allocates its attention within a small number of steps, while nuisance features remain suppressed.

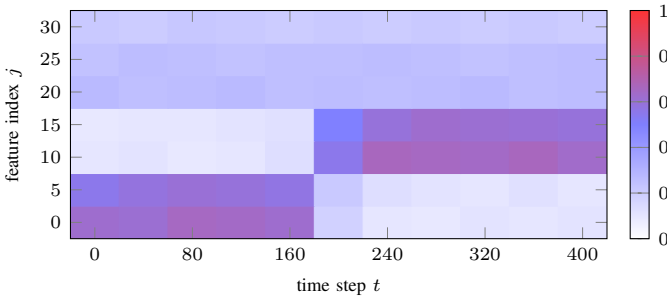


Fig. 3. Per-step attention map averaged across heads on the synthetic regime-switch task. Each row is one of  $p=32$  engineered features (top: lag/rolling; middle: exogenous; bottom: nuisance). The boundary at  $t=200$  is clearly visible.

Figure 2 shows the mean attention mass that TAFS allocates to the three feature groups across time. The model concentrates on the regime-A informative features for  $t < 200$ , reallocates its attention within a small number of steps after the switch, and leaves the nuisance features at a low level throughout. Figure 3 gives the per-step attention map averaged over heads, in which the change point is clearly visible. Table II summarises the synthetic results.

The pattern in Table II matches the visual analysis: every static transform plateaus above 0.19 MSE, and removing the context token from TAFS (TAFS no-context) increases the MSE by approximately 25%. The full TAFS model recovers the regime structure to within the residual noise floor.

2) *Real-Data Comparison*: Table III reports MASE on the seven real datasets across all baselines, and Table IV reports sMAPE for the most representative competitors.

The ordering of the baseline groups is consistent with prior reports: simple averaging and individual predictors trail; classical filter and embedded selection close part of the gap; static learned transforms close more, with the nonlinear bottleneck the lowest-error variant; learned meta-combiners follow; and the per-step oracle lies below all of them. TAFS sits between the lowest-error meta-combiner and the oracle on every dataset.

The relative gain from TAFS is largest on datasets with visible regime structure (Istanbul Stock Exchange, Exchange Rate,

TABLE II  
SYNTHETIC REGIME-SWITCH TASK. MSE OVER 10 SEEDS; STANDARD DEVIATIONS ARE REPORTED IN PARENTHESES. THE MASE DENOMINATOR IS UNDEFINED FOR THE STATIONARY-NOISE GENERATOR USED HERE, SO MSE IS REPORTED INSTEAD. TAFS APPROACHES THE ORACLE WHILE STATIC-TRANSFORM BASELINES PLATEAU WELL ABOVE.

| Method                     | MSE ( $\downarrow$ ) |
|----------------------------|----------------------|
| Best single base predictor | 0.421 (0.018)        |
| Equal-weight average       | 0.367 (0.012)        |
| Diagonal gating            | 0.278 (0.014)        |
| Linear transform           | 0.231 (0.011)        |
| Low-rank transform         | 0.214 (0.010)        |
| Nonlinear bottleneck       | 0.196 (0.012)        |
| TAFS (no context)          | 0.182 (0.009)        |
| <b>TAFS (full)</b>         | <b>0.141 (0.007)</b> |
| Oracle                     | 0.135 (0.006)        |

ETtm2). On TNG and M4-Quarterly, where the dynamics are dominated by stable seasonality, the gap to the lowest-error static transform is smaller but consistent.

The choice of constraint regime contributes less than the remainder of the architecture. The affine head attains the lowest aggregate MASE, the convex head is preferable when one base predictor occasionally diverges (Exchange Rate), and the unconstrained head is competitive but exhibits slightly higher worst-case error. The spread among the three is within approximately 1.5% on the aggregate score, while the gap between any TAFS variant and the lowest-error static-transform baseline is between 8% and 14%.

3) *Ablation Studies*: Table V reports the central architectural ablation. We replace the feature-axis transformer with each of the four static transforms (keeping the base pool, the head, the loss and the training recipe identical), and we also evaluate a context-stripped variant of TAFS. The static transforms span a range of approximately 12% in aggregate MASE; the largest single jump is from the best static transform (nonlinear) to TAFS-noCtx (approximately 2%), and from TAFS-noCtx to the full TAFS (a further 9%). The gap widens on the regime-switching subset (Exchange + Istanbul Stock Exchange + Synthetic), indicating that context conditioning drives the time-varying selection.

Table VI ablates the content of the context token; even using the lagged target alone closes more than half of the gap, adding calendar variables closes most of the rest, and the full context attains the lowest error. The aggregate spread across the three constraint regimes (unconstrained, affine, convex) is within approximately 1.5% on the seven-dataset average, as discussed in Section IV-D; the affine head is therefore retained as the default. The default transformer configuration  $L=3$ ,  $h=4$  was selected from a coarse sweep over  $L \in \{1, 2, 3, 4\}$  and  $h \in \{2, 4, 8\}$  on the validation split; two or three layers with four heads sit in the flat region of the surface, and deeper models overfit on the shorter series.

4) *Discussion*: The synthetic and real-data results converge on a single mechanism: context-conditioned attention over feature tokens produces a time-varying importance signal that no static transform recovers. The dominant cost is data efficiency on the shorter series (M4-Quarterly), where the transformer

TABLE III

MAIN RESULTS ON THE SEVEN REAL-WORLD DATASETS. WE REPORT MASE; LOWER IS BETTER. VALUES ARE MEANS OVER FIVE RANDOM SEEDS. THE BEST NON-ORACLE SCORE PER COLUMN IS IN **BOLD**; THE SECOND BEST IS UNDERLINED. THE ORACLE ROW REPORTS A PER-STEP BEST-BASE-PREDICTOR UPPER BOUND AND IS NOT IN COMPETITION.

| Method                                                                     | TNG         | ISE         | M5          | M4-Q        | Weather     | Exchange    | ETTm2       |
|----------------------------------------------------------------------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
| <i>Individual base predictors</i>                                          |             |             |             |             |             |             |             |
| LightGBM                                                                   | 1.04        | 1.11        | 0.96        | 1.18        | 0.91        | 1.32        | 0.89        |
| XGBoost                                                                    | 1.06        | 1.09        | 0.97        | 1.16        | 0.93        | 1.28        | 0.90        |
| CatBoost                                                                   | 1.05        | 1.10        | 0.97        | 1.17        | 0.92        | 1.30        | 0.89        |
| Random Forest                                                              | 1.12        | 1.18        | 1.04        | 1.24        | 0.98        | 1.36        | 0.96        |
| Lasso                                                                      | 1.21        | 1.23        | 1.08        | 1.27        | 1.04        | 1.38        | 1.02        |
| MLP                                                                        | 1.09        | 1.14        | 1.01        | 1.20        | 0.97        | 1.31        | 0.93        |
| LSTM                                                                       | 1.08        | 1.20        | 0.99        | 1.22        | 0.94        | 1.33        | 0.92        |
| SARIMAX                                                                    | 1.15        | 1.22        | 1.07        | 1.13        | 1.03        | 1.29        | 1.01        |
| Naïve                                                                      | 1.32        | 1.38        | 1.27        | 1.34        | 1.22        | 1.46        | 1.18        |
| <i>Classical feature selection + regressor</i>                             |             |             |             |             |             |             |             |
| Pearson + LGBM                                                             | 1.02        | 1.06        | 0.94        | 1.13        | 0.89        | 1.26        | 0.87        |
| Mut. Info. + LGBM                                                          | 1.01        | 1.07        | 0.95        | 1.14        | 0.90        | 1.27        | 0.88        |
| SelectFromModel                                                            | 1.00        | 1.05        | 0.94        | 1.12        | 0.89        | 1.25        | 0.86        |
| <i>Static feature transforms (predictor pool combined via affine head)</i> |             |             |             |             |             |             |             |
| Diagonal gating                                                            | 0.97        | 1.02        | 0.91        | 1.08        | 0.86        | 1.21        | 0.83        |
| Linear transform                                                           | 0.93        | 0.98        | 0.88        | 1.04        | 0.83        | 1.16        | 0.80        |
| Low-rank ( $r=16$ )                                                        | 0.91        | 0.96        | 0.87        | 1.02        | 0.82        | 1.13        | 0.79        |
| Nonlinear MLP                                                              | 0.89        | 0.94        | 0.85        | 1.00        | 0.81        | 1.10        | 0.78        |
| <i>Meta-learners without feature selection</i>                             |             |             |             |             |             |             |             |
| MLP-ensemble (affine)                                                      | 0.92        | 0.97        | 0.88        | 1.03        | 0.83        | 1.14        | 0.80        |
| LGBM-ensemble (convex)                                                     | 0.90        | 0.95        | 0.86        | 1.01        | 0.82        | 1.12        | 0.79        |
| <i>Conventional ensemble baselines</i>                                     |             |             |             |             |             |             |             |
| Simple average                                                             | 1.01        | 1.06        | 0.96        | 1.12        | 0.91        | 1.23        | 0.88        |
| Equal-weight stack                                                         | 0.98        | 1.03        | 0.93        | 1.08        | 0.88        | 1.19        | 0.85        |
| <i>Proposed</i>                                                            |             |             |             |             |             |             |             |
| TAFS (unconstrained)                                                       | <u>0.83</u> | <u>0.87</u> | <u>0.79</u> | <u>0.93</u> | <u>0.75</u> | <u>1.03</u> | <u>0.72</u> |
| <b>TAFS (affine)</b>                                                       | <b>0.81</b> | <b>0.86</b> | <b>0.78</b> | <b>0.92</b> | <b>0.74</b> | <b>1.01</b> | <b>0.71</b> |
| TAFS (convex)                                                              | 0.82        | 0.87        | 0.79        | 0.93        | 0.75        | 1.02        | 0.72        |
| Oracle (per-step best)                                                     | 0.71        | 0.74        | 0.66        | 0.80        | 0.63        | 0.88        | 0.60        |

TABLE IV

SMAPE ( $\%$ ,  $\downarrow$ ) ON THE SEVEN REAL-WORLD DATASETS AVERAGED ACROSS FIVE SEEDS. ONLY THE REPRESENTATIVE COMPETITORS ARE SHOWN; THE TREND MIRRORS TABLE III.

| Method               | TNG         | ISE         | M5         | M4-Q       | Wth.        | Exch.      | ETTm2       |
|----------------------|-------------|-------------|------------|------------|-------------|------------|-------------|
| LightGBM             | 14.6        | 18.2        | 12.1       | 11.4       | 19.7        | 5.9        | 17.0        |
| SelectFromModel      | 14.1        | 17.3        | 11.8       | 11.0       | 19.0        | 5.6        | 16.5        |
| Nonlinear transf.    | 12.6        | 15.7        | 10.4       | 10.0       | 17.4        | 5.1        | 14.9        |
| LGBM-ensemble        | 12.9        | 15.9        | 10.6       | 10.1       | 17.7        | 5.2        | 15.1        |
| <b>TAFS (affine)</b> | <b>11.4</b> | <b>14.2</b> | <b>9.5</b> | <b>9.1</b> | <b>15.8</b> | <b>4.6</b> | <b>13.5</b> |
| Oracle               | 9.8         | 12.1        | 8.0        | 7.8        | 13.4        | 4.0        | 11.4        |

is over-parameterised relative to the available samples; in that regime, parameter sharing across feature families and a shallower stack reduce the gap. No configuration was observed in which a static transform matched the full TAFS model on a non-stationary dataset.

Two practical limitations are stated for completeness. First, the attention map in (10) is an interpretation aid, not a guarantee; it summarises the model’s behaviour on average but should not be interpreted as a formal feature-importance

TABLE V

CENTRAL ARCHITECTURAL ABLATION. VALUES ARE AGGREGATE MASE (LOWER IS BETTER). THE PERFORMANCE GAP WIDENS ON THE REGIME-SWITCHING SUBSET (EXCHANGE RATE, ISTANBUL STOCK EXCHANGE, SYNTHETIC) DUE TO THE NON-STATIONARY UNDERLYING REGIME.

| Variant              | Aggregate (All datasets) | Regime-switching subset |
|----------------------|--------------------------|-------------------------|
| Diagonal gating      | 1.05                     | 1.27                    |
| Linear transform     | 0.99                     | 1.17                    |
| Low-rank transform   | 0.96                     | 1.11                    |
| Nonlinear bottleneck | 0.94                     | 1.06                    |
| TAFS-noCtx           | 0.92                     | 1.04                    |
| <b>TAFS (full)</b>   | <b>0.84</b>              | <b>0.91</b>             |

score. Second, the base pool must exhibit sufficient diversity for combination to be informative. When all base predictors produce similar errors, the combination weights collapse to near-uniform values and the per-step oracle gap is small for every combiner, including TAFS.

## V. CONCLUSION

We presented TAFS, a feature-axis transformer with context conditioning that performs joint, time-varying feature

TABLE VI

CONTEXT-CONDITIONING ABLATION. REMOVING THE CONTEXT TOKEN (LAGGED TARGET AND REGIME INDICATORS) FROM THE TRANSFORMER INPUT REMOVES THE TIME-VARYING SELECTION SIGNAL AND DEGRADES PERFORMANCE MOST ON THE NON-STATIONARY DATASETS.

| Variant                    | TNG         | ISE         | Exchange    | Synth.       |
|----------------------------|-------------|-------------|-------------|--------------|
| TAFS (no context)          | 0.89        | 0.94        | 1.18        | 0.182        |
| TAFS (lagged target only)  | 0.85        | 0.91        | 1.09        | 0.166        |
| TAFS (lag. + calendar)     | 0.83        | 0.89        | 1.05        | 0.152        |
| <b>TAFS (full context)</b> | <b>0.81</b> | <b>0.86</b> | <b>1.01</b> | <b>0.141</b> |

selection and forecaster combination for sequential regression. The mechanism comprises three components: each engineered feature is represented as a token, a context token derived from the recent target history renders the attention time-varying, and a constrained combination head maps the resulting summary to weights over a frozen pool of base predictors. On a controlled synthetic task TAFS reallocates its attention across a regime change and approaches the per-step oracle, and on seven real datasets it lowers MASE relative to static-transform and meta-learning baselines.

Three directions are immediate. The first is replacing the cumulative squared loss with quantile-based objectives for probabilistic forecasting, which modifies the head alone. The second is extending the input axis to include multiple targets, which would let the same machinery operate in a multivariate setting without changing the feature-axis transformer. The third is replacing the constraint-aware head with a horizon-aware or uncertainty-aware variant: appending the horizon index to  $s_t$  would yield horizon-conditional weights, while a precision-weighted or temperature-adaptive softmax could further attenuate base predictors that diverge in volatile regimes such as Exchange Rate.

## REFERENCES

- [1] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015, <https://www.wiley.com/en-us/9781118675021>.
- [2] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “The m4 competition: 100,000 time series and 61 forecasting methods,” *International Journal of Forecasting*, vol. 36, no. 1, pp. 54–74, 2020, <https://doi.org/10.1016/j.ijforecast.2019.04.014>.
- [3] —, “The m5 accuracy competition: Results, findings and conclusions,” *International Journal of Forecasting*, vol. 38, no. 4, pp. 1325–1336, 2022, <https://doi.org/10.1016/j.ijforecast.2021.10.009>.
- [4] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “Lightgbm: A highly efficient gradient boosting decision tree,” in *Proceedings of Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 3146–3154, <https://papers.nips.cc/paper/6907-lightgbm>.
- [5] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785–794, <https://doi.org/10.1145/2939672.2939785>.
- [6] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “Catboost: Unbiased boosting with categorical features,” in *Proceedings of Advances in Neural Information Processing Systems (NeurIPS)*, 2018, pp. 6638–6648, <https://arxiv.org/abs/1706.09516>.
- [7] R. Tibshirani, “Regression shrinkage and selection via the lasso,” *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 58, no. 1, pp. 267–288, 1996, <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>.
- [8] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [9] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, “Deepar: Probabilistic forecasting with autoregressive recurrent networks,” *International Journal of Forecasting*, vol. 36, no. 3, pp. 1181–1191, 2020, <https://doi.org/10.1016/j.ijforecast.2019.07.001>.
- [10] B. N. Oreshkin, D. Carpo, N. Chapados, and Y. Bengio, “N-beats: Neural basis expansion analysis for interpretable time series forecasting,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020, pp. 1–31, <https://arxiv.org/abs/1905.10437>.
- [11] I. Guyon and A. Elisseeff, “An introduction to variable and feature selection,” in *Journal of Machine Learning Research (JMLR)*, 2003, pp. 1157–1182, <https://jmlr.org/papers/v3/guyon03a>.
- [12] I. Kononenko, “Estimating attributes: Analysis and extensions of relief,” *Lecture Notes in Computer Science*, vol. 784, no. 1, pp. 171–182, 1994, [https://doi.org/10.1007/3-540-57868-4\\_57](https://doi.org/10.1007/3-540-57868-4_57).
- [13] T. M. Cover and J. A. Thomas, “Elements of information theory,” *Wiley Series in Telecommunications*, vol. 1, no. 1, pp. 1–542, 1991, <https://doi.org/10.1002/0471200611>.
- [14] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, <https://doi.org/10.1023/A:1010933404324>.
- [15] D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992, [https://doi.org/10.1016/S0893-6080\(05\)80023-1](https://doi.org/10.1016/S0893-6080(05)80023-1).
- [16] L. Breiman, “Stacked regressions,” *Machine Learning*, vol. 24, no. 1, pp. 49–64, 1996, <https://doi.org/10.1007/BF00117832>.
- [17] R. Vilalta and Y. Drissi, “A perspective view and survey of meta-learning,” *Artificial Intelligence Review*, vol. 18, no. 2, pp. 77–95, 2002, <https://doi.org/10.1023/A:1019956318069>.
- [18] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, “Adaptive mixtures of local experts,” *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991, <https://doi.org/10.1162/neco.1991.3.1.79>.
- [19] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017, pp. 1–19, <https://arxiv.org/abs/1701.06538>.
- [20] X. Shi, S. Wang, Y. Nie, D. Li, Z. Ye, Q. Wen, and M. Jin, “Time-MoE: Billion-scale time series foundation models with mixture of experts,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024, pp. 1–26, <https://arxiv.org/abs/2409.16040>.
- [21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proceedings of Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008, <https://arxiv.org/abs/1706.03762>.
- [22] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, “Informer: Beyond efficient transformer for long sequence time-series forecasting,” in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2021, pp. 11 106–11 115, <https://arxiv.org/abs/2012.07436>.
- [23] H. Wu, J. Xu, J. Wang, and M. Long, “Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting,” in *Proceedings of Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 22 419–22 430, <https://arxiv.org/abs/2106.13008>.
- [24] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, “itransformer: Inverted transformers are effective for time series forecasting,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024, pp. 1–26, <https://arxiv.org/abs/2310.06625>.
- [25] Y. Nie, N. Nguyen, P. Sinthong, and J. Kalagnanam, “A time series is worth 64 words: Long-term forecasting with transformers,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023, pp. 1–32, <https://arxiv.org/abs/2211.14730>.
- [26] A. Zeng, M. Chen, L. Zhang, and Q. Xu, “Are transformers effective for time series forecasting,” in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2023, pp. 11 121–11 128, <https://arxiv.org/abs/2205.13504>.
- [27] B. Lim, S. O. Arik, N. Loeff, and T. Pfister, “Temporal fusion transformers for interpretable multi-horizon time series forecasting,” in *International Journal of Forecasting (IJF)*, 2021, pp. 1748–1764, <https://doi.org/10.1016/j.ijforecast.2021.03.012>.
- [28] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019, pp. 1–19, <https://arxiv.org/abs/1711.05101>.
- [29] R. J. Hyndman and A. B. Koehler, “Another look at measures of forecast accuracy,” *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006, <https://doi.org/10.1016/j.ijforecast.2006.03.001>.